- گزارش کار فاز تمرین سری پنجم درس MicroBlaze : FPGA
- فاز دوم
- اعضای گروه : عرفان سائنی_۴۰۰۲۳۰۳۶ ، پدرام شیخ علیزاده_۴۰۰۲۳۰۴۷

❖ مقدمه :

در این تمرین قصد داریم، با استفاده از نرم افزار HLS در Vivado، ماژول PacketChecker خود را که به زبان c توصیف کردیم، به یک IP core قابل استفاده در vivado تبدیل کنیم و ماژول UART را که در قسمت های قبلی پیاده سازی کردیم به آن متصل کنیم و نتایج را مشاهده کنیم.

❖ توصیف PacketChecker با زبان c :

قسمت های خالی برنامه داده شده را با کد مناسب پر میکنیم :

بخش rst:

```
if (clk)
{

    out.error = false;
    out.wrEn = false;
    out.rdEn = false;
    out.dataOutRdy = false;

    if (rst)
    {
        //*** TODO ***//
        // Reset all signals to zero
        count_packets = 0;
        count_response = 0;
        out.Packet_state = ND;
        out.Response_state = IDLE;
    }
```

در صورت ریست شدن تمام حالت ها به حالت اولیه بازمیگردند. همچنین سیگنال های کنترلی مورد نظر را تک کلاکه میکنیم.

بخش Packet_state :

```
case ND:
    // Look for header
    //*** TODO ***//
    tmp_checksum = 0;
    count_packets = 0;
    if (dataIn == HEADER)
    {
        out.Packet_state = HDR;
        tmp_checksum += dataIn;
    }

    break;
```

در حالت ND، در هر لبه بالارونده اگر HEADER را دریافت کردیم، به حالت بعدی، یعنی HDR میرویم. همچنین در هر مرحله tmp_checksum را با دیتای ورودی جمع میزنیم تا در آخر ارور Checksum را بررسی کنیم.

```
case HDR:

    // Look for Command
    //*** TODO ***//
    if (dataIn == COMMWRITE)
    {
        out.Packet_state = WR;
        tmp_checksum += dataIn;
    }
    else if (dataIn == COMMREAD)
    {
        out.Packet_state = RD;
        tmp_checksum += dataIn;
    }
    else
    {
        out.error = true;
        printf("the command is not valid !");
    }

    break;
```

در حالت HDR ، در صورت دریافت هرکدام از دستورات دریافت شده، به حالت WR یا RD میرویم. در غیر این صورت، ارور را به خروجی منتقل میکنیم.

```
case WR:

    //*** TODO ***//

    switch (count_packets)
    {
    case 0:
        out.wrAddress = dataIn;
        out.wrAddress = out.wrAddress << 8;
        tmp_checksum += dataIn;
        count_packets += 1;
        break;

    case 1:
        out.wrAddress += dataIn;
        tmp_checksum += dataIn;
        count_packets += 1;
        break;
```

در حالت WR، با استفاده از count_packets مشخص میکنیم که در چه مرحله ای هستیم. با استفاده از عملگر

شیفت(<< یا >>) میتوانیم اعداد باینری را به میزان دلخواه شیفت بدهیم. از این طریق آدرس ها و دیتا هارا در هر مرحله در جای مناسب ذخیره میکنیم.

ذخیره دیتا ها :

```
case 2:
    out.wrData += (uint32_t)dataIn << 24;
    tmp_checksum += dataIn;
    count_packets += 1;
    break;

case 3:
    out.wrData += (uint32_t)dataIn << 16;
    tmp_checksum += dataIn;
    count_packets += 1;
    break;

case 4:
    out.wrData += (uint32_t)dataIn << 8;
    tmp_checksum += dataIn;
    count_packets += 1;
    break;

case 5:
    out.wrData += (uint32_t)dataIn;
    tmp_checksum += dataIn;
    count_packets += 1;
    break;
```

```
case 6:
    checksumH = dataIn;
    count_packets += 1;
    break;

case 7:
    checksumL = dataIn;
    count_packets += 1;
    break;

case 8:
    out.Packet_state = ND;

    if (tmp_checksum == (((uint16_t)checksumH << 8) + (uint16_t)checksumL))
    {
        out.wrEn = true;
    }
    else
    {
        out.error = true;
        printf("CheckSum error !");
    }
    break;
}
```

در آخر چک سام هارا در دو متغیر در هر مرحله ذخیره میکینم و در کلاک آخر، در صورتی که مقدار checksum دریافت شده با مقدار محاسبه شده ما برابر باشد، wren فعال میشود، و در غیر این صورت، ارور به خروجی میدهد.

همچنین در این مرحله به حالت اولیه بازمیگردیم.

حالت RD :

مشابه قسمت WR، آدرس ها و چک سام ها ذخیره میشوند، و در آخر داریم :

```
case 4:
    out.Packet_state = ND;

    if (tmp_checksum == (((uint16_t)checksumH << 8) + (uint16_t)checksumL))
    {
        out.rdEn = true;
    }
    else
    {
        out.error = true;
        printf("CheckSum error !");
    }
    break;
```

در صورت نداشتن ارور چک سام، rdEn فعال میشود.

قسمت Response_state :

این قسمت باید خارج از بلوک DataInRdy قرار بگیرد.

```
case IDLE:

    checksum = 0x00;
    count_response = 0;
    if (rdRdy == true)
    {
        // Initialize sending packets in given order
        //*** TODO ***//
        out.Response_state = COMMAND;
        out.dataOut = HEADER;
        out.dataOutRdy = true;
        checksum += HEADER;
    }
    break;

case COMMAND:
    // Send Command packet and get data packets ready to be sent
    //*** TODO ***//
    out.Response_state = DATA;
    out.dataOut = COMMREAD;
    out.dataOutRdy = true;
    checksum += COMMREAD;
    break;
```

در هر کلاک در صورت دریافت rdRdy، به حالت بعدی میرویم و پروسه انتقال دیتای ۳۲ بیتی دریافت شده شروع میشود. در هر مرحله مقدار دیتای فرستاده شده با checksum جمع میشود تا در آخر آن را انتقال دهیم.

```
case DATA:
    // Send Data packets and Calculate CheckSum
    //*** TODO ***//
    switch (count_response)
    {
    case 0:
        out.dataOut = (rdData >> 24) & 0xFF;
        out.dataOutRdy = true;
        checksum += (rdData >> 24) & 0xFF;
        count_response++;
        break;

    case 1:
        out.dataOut = (rdData >> 16) & 0xFF;
        out.dataOutRdy = true;
        checksum += (rdData >> 16) & 0xFF;
        count_response++;
        break;

    case 2:
        out.dataOut = (rdData >> 8) & 0xFF;
        out.dataOutRdy = true;
        checksum += (rdData >> 8) & 0xFF;
        count_response++;
        break;
```

با شیفت، و سپس AND کردن دیتای دریافت شده، میتوانیم آن را به ترتیب از بخش High به Low انتقال دهیم.

انتقال checksum:

```
case CHKSUM:
    // Send CheckSum packets and reset variables
    //*** TODO ***//
    switch (count_response)
    {
    case 4:
        out.dataOut = (checksum >> 8) & 0xFF;
        out.dataOutRdy = true;
        count_response++;
        break;

    case 5:
        out.dataOut = checksum & 0xFF;
        out.dataOutRdy = true;
        out.Response_state = IDLE;
        break;
    }
    break;
```

تست برنامه توسط C :

```c
int main()
    for (int i = 0; i < 30; i++)
    {
        if (i < 12) //* write command test *//
        {
            out3 = PacketProcess(clock, reset, InputData1[i], InputReady, ReadData, ReadReady);
            printf("\n");
            printf("state : %d\n", out3.Packet_state);
            printf("count packets : %d\n", count_packets);
            printf("wrAddress: %x\n", out3.wrAddress);
            printf("wrData: %x\n", out3.wrData);
            printf("checksum: %x\n", ((uint16_t)checksumH << 8) + (uint16_t)checksumL);
            printf("cal_checksum: %x\n", tmp_checksum);
            printf("WrEn: %d\n", out3.wrEn);
        }
        else if (i >= 12 && i < 20) //* read command test *//
        {
            out3 = PacketProcess(clock, reset, InputData2[i - 12], InputReady, ReadData, ReadReady);
            printf("\n");
            printf("state : %d\n", out3.Packet_state);
            printf("count packets : %d\n", count_packets);
            printf("rdAddress: %x\n", out3.rdAddress);
            printf("rdEn: %d\n", out3.rdEn);
            printf("checksum: %x\n", ((uint16_t)checksumH << 8) + (uint16_t)checksumL);
            printf("cal_checksum: %x\n", tmp_checksum);
        }
        else if (i >= 20) //* read data test *//
```

تمام حالات را چک میکنیم، نتایج کاملا درست و مقبول است :

بخش WR :

```
state : 3
count packets : 7
wrAddress: 614
wrData: ff550121
checksum: 100
cal_checksum: 1cb
WrEn: 0

state : 3
count packets : 8
wrAddress: 614
wrData: ff550121
checksum: 1cb
cal_checksum: 1cb
WrEn: 0

state : 0
count packets : 8
wrAddress: 614
wrData: ff550121
checksum: 1cb
cal_checksum: 1cb
WrEn: 1
```

بخش RD:

```
state : 4
count packets : 3
rdAddress: 614
rdEn: 0
checksum: 1cb
cal_checksum: 154

state : 4
count packets : 4
rdAddress: 614
rdEn: 0
checksum: 154
cal_checksum: 154

state : 0
count packets : 4
rdAddress: 614
rdEn: 1
checksum: 154
cal_checksum: 154
```

بخش RdData :

```
states : 0 , 2
rdRdy : 0
count rsponse : 2
dataOut : 34
dataOutRdy : 1

states : 0 , 2
rdRdy : 0
count rsponse : 3
dataOut : 56
dataOutRdy : 1

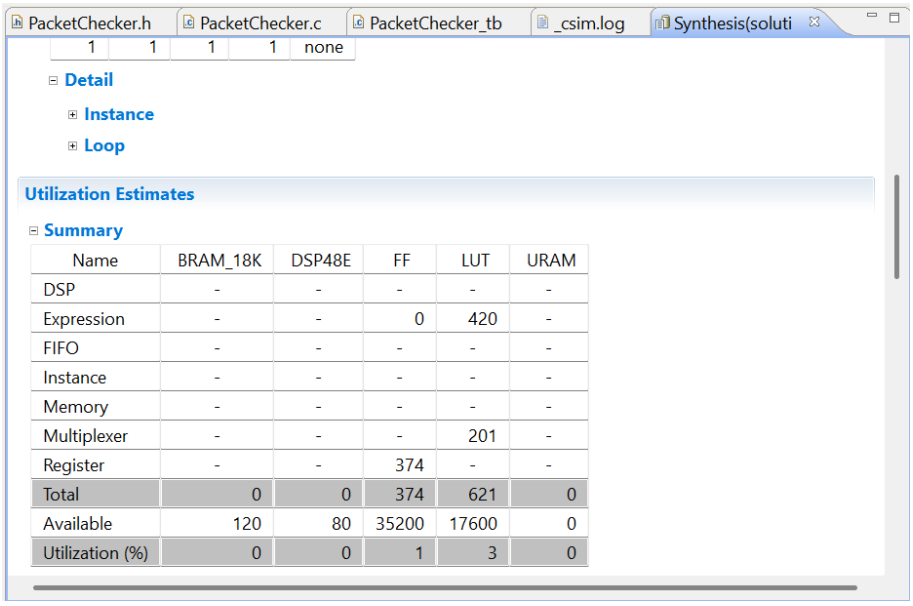
states : 0 , 3
rdRdy : 0
count rsponse : 4
dataOut : 78
dataOutRdy : 1

states : 0 , 3
rdRdy : 0
count rsponse : 5
dataOut : 2
dataOutRdy : 1

states : 0 , 0
rdRdy : 0
count rsponse : 5
dataOut : 4e
dataOutRdy : 1
```

❖ ساخت IP core در نرم افزارHLS

به طریق گفته شده کد خود را به نرم افزار میدهیم. نتیجه C Synthesises :

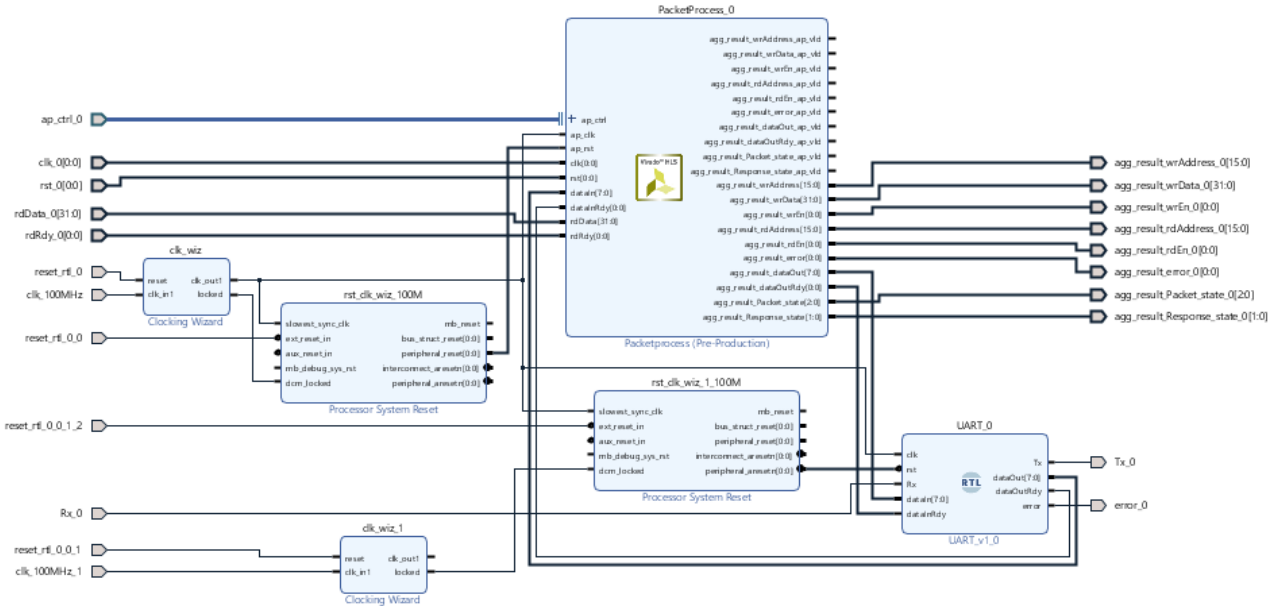


Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	420	-
FIFO	-	-	-	-	-
Instance	-	-	-	-	-
Memory	-	-	-	-	-
Multiplexer	-	-	-	201	-
Register	-	-	374	-	-
Total	0	0	374	621	0
Available	120	80	35200	17600	0
Utilization (%)	0	0	1	3	0

در نهایت با rtl export ماژول خودرا میسازیم.

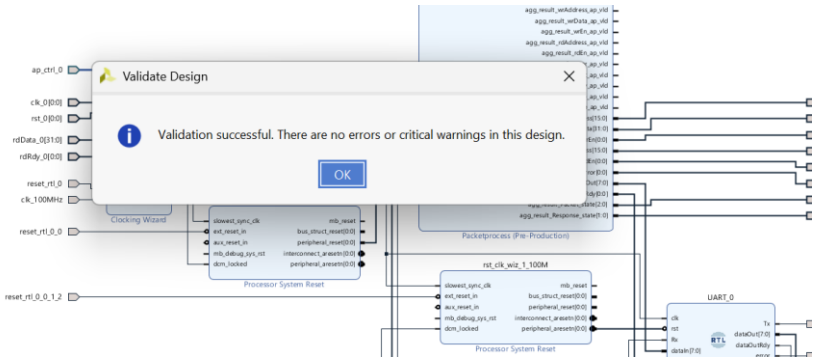
❖ اضافه کردن IP core به برنامه :

ماژول UARTرا به همراه آی پی کور ساخته شده در Block design به یکدیگر متصل میکنیم. از طریق گزینه Automation، پورت های ریست و کلاک به یک منبع مناسب متصل میشوند.



همچنین پورت های ورودی و خروجی تاپ ماژول خودرا به صورت external تعریف میکنیم.

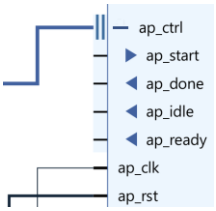
نتیجه Validate Design :



در نهایت با گزینه HDL Wrapper ماژول خود را میسازیم.

❖ ایجاد تست بنچ و شبیه سازی :

چند نکته درباره سیگنال های کنترلی IP core ساخته شده:



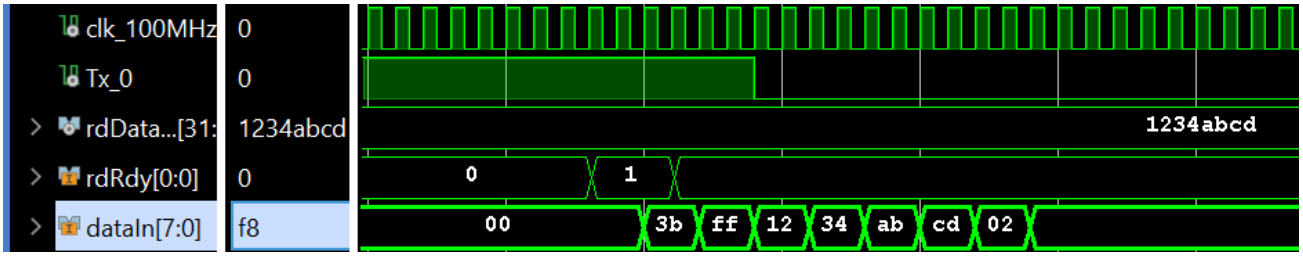
پورت ap_start : در صورت صفر بودن این ورودی، IP core غیرفعال است و خروجی undefined میشود.

پورت ap_rst : در صورت یک بودن این ورودی، ماژول ما تغییر حالت نمیدهد، و همواره در حالت اولیه باقی میماند. لذا خروجی های ما همواره صفر میشود.

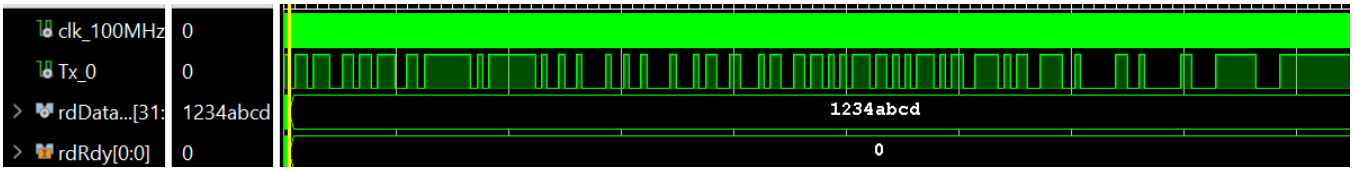
در تست بنچ، ابتدا این دو ورودی را تنظیم میکنیم : به ap_start همواره یک میدهیم، و پورت ap_rst را از طریق ماژول Processor System Reset متصل به آن همواره صفر میکنیم:

```
Rx_0<='1';
rdData_0<=x"1234abcd";
rdRdy_0<="0";
ap_ctrl_0_start<='1';
reset_rtl_0<='0';
reset_rtl_0_0<='1';
reset_rtl_0_0_1<='0';
reset_rtl_0_0_1_2<='0';
rst_0<="0";
wait for clk_period*200;
```

تست rdData :



با یک شدن rdRdy، در هر کلاک دیتا ها به صورت پکت تعریف شده فرستاده میشوند و در بافر FIFO تعریف شده در UART، ذخیره میشوند. سپس توسط Tx فرستاده میشوند :



تست Wr :



خطوط اول و دوم، به ترتیب WrAddress، WrData هستند و دو خط بعدی ورودی های dataIn و dataInRdy هستند.

با فعال شدن dataInRdy ، هر پکت ارسالی توسط IP core دریافت میشوند، و به خروجی های مناسب آدرس و دیتا داده میشوند.

❖ تقسیم کار گروه :

تمامی مراحل پروژه با همکاری یکدیگر زده شده و بخش خاصی به دیگری واگذار نشده